
Building AI Agents That Replaced ~95% of Manual QA Effort

How a Senior Director rebuilt Quality Engineering at AutoRabit around AI platforms and engineering intelligence systems that serve Product, Engineering, QA, SRE, and Support — automating ~95% of manual effort and reducing customer escalations by ~70%.

BY

Madhu Sudhan Painoori

Senior Director, Quality Engineering — AutoRabit Pvt Ltd

p.mschary@gmail.com | +91 80086 90004 | Hyderabad, India

EXECUTIVE SUMMARY

This playbook documents how Quality Engineering at AutoRabit — a DevSecOps SaaS platform for Salesforce — was rebuilt around AI agents and an engineering-intelligence layer that serves not just QA, but Product, Engineering, SRE, and Support. It is written for engineering leaders who are being asked to do more with AI across the organization and want a candid account from someone who has actually shipped it.

~95% MANUAL EFFORT AUTOMATED VIA AI AGENTS	~70% CUSTOMER ESCALATIONS REDUCED	110+ ENGINEERS IN MATRIX SCOPE	4 SAAS PRODUCTS UNDER QE OWNERSHIP
--	---	--	--

The Headline

Manual, repetitive work — test design, regression authoring, impact analysis, environment validation, recruiting screen-outs, performance triage, release reporting, engineering metric collection — collapsed by approximately 95% across our portfolio. Customer escalations fell by approximately 70%, with measurable improvement in churn signals tied to quality and deliverables. We did this with a 10-person QA org influencing a 100+ engineer matrix that spans Product, Engineering, QA, SRE, and Support, across four product lines.

How We Got There

The transformation rested on four moves, executed in sequence:

- 1. **Tool portfolio decision.** We adopted nine AI and GenAI tools — Copilot, ChatGPT, Claude, Amazon Bedrock, AWS Kiro, Cursor, KaneAI, Mabl, and TestAI — each chosen for a specific use case rather than as a blanket rollout.
- 2. **Agentic AI platform.** We built a flagship platform on AWS Kiro that consumes Jira stories, drives story development, runs impact analysis on code changes, generates the test suite, and executes it through Playwright + MCP with Amazon Bedrock for AI analysis and self-healing — closing the loop from requirement to validated release.
- 3. **Specialized agents serving the broader engineering org.** Performance Optimizing Agent for profiling and scalability, PR Quality & Coverage Agent activating on every pull request to lift unit-test coverage and code quality across the engineering team, and AutoScreen for recruitment drive efficiency.
- 4. **RabbitView and the in-house intelligence layer.** We built RabbitView — the engineering intelligence platform for Product, Engineering, and QA monitoring (DORA metrics, individual and team velocity, product backlog, release monitoring, team analytics) — alongside Report Portal, an AI-enabled TMS (Bedrock-powered duplicate / invalid-test detection), Telemetry, and Skill Matrix. All on TypeScript, Python, Docker, PostgreSQL, and Redis.

A NOTE ON WHAT THIS PLAYBOOK IS NOT

This is not an AI vendor pitch. It is not a claim that AI replaces engineers. It is a working account of how a Senior Director combined commercial AI tooling, a custom agentic platform, and in-house tooling to remove the most painful 95% of QA toil — and what we learned along the way.

MADHU SUDHAN PAINOORI

Senior Director, Quality Engineering · AutoRabit · AI Platforms Serving Product · Engineering · QA · SRE · Support

AI-NATIVE ENGINEERING EXCELLENCE

Cross-Functional Architecture · Agent Fleet · Intelligence Layer

~95%

MANUAL EFFORT AUTOMATED

via AI agents across QA workflows

~70%

CUSTOMER ESCALATIONS REDUCED

with reduced churn tied to quality

110+

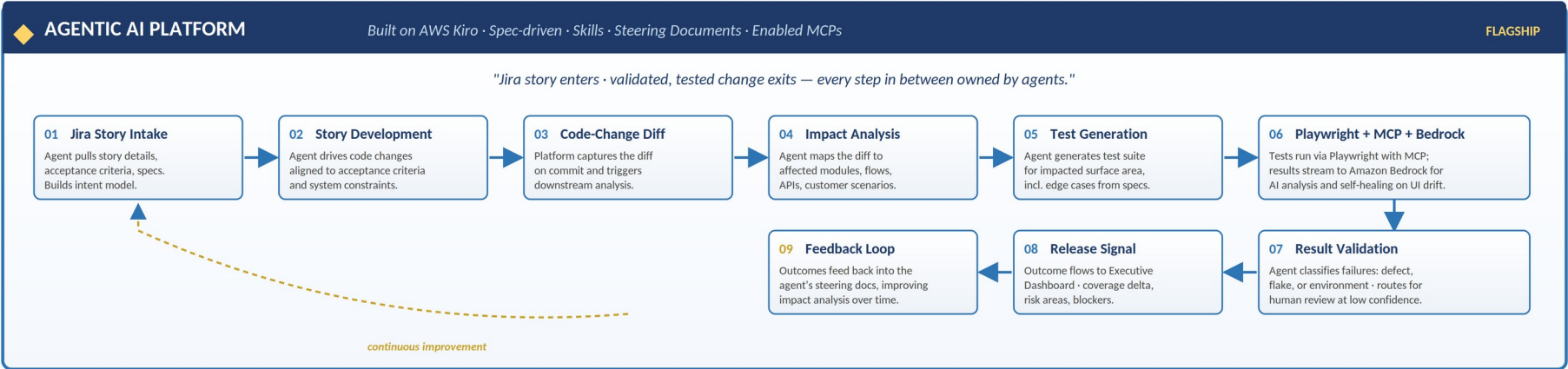
CROSS-FUNCTIONAL SCOPE

Product · Eng · QA · SRE · Support

4

SAAS PRODUCTS UNDER QE OWNERSHIP

Release Mgmt · CodeScan · Vault · DataOps



AI TOOL FLEET

commercial · adopted

Copilot

ChatGPT

Claude

Cursor

Amazon Bedrock

AWS Kiro

Playwright + MCP

KaneAI

Mabl

TestAI

SELECTION DOCTRINE

Each tool chosen for a specific job — IDE assistance, long-context reasoning, enterprise hosting, spec-driven agent dev, low-code UI flows, self-healing automation.

SPECIALIZED AGENT FLEET

in-house · purpose-built

Performance Optimizing Agent

Profiling on demand · optimization proposals · scalability assessment. Shifts performance work left across engineering.

PR Quality & Coverage Agent

Activates on every PR · identifies coverage gaps · recommends test additions · assesses code quality. Serves entire eng team.

AutoScreen — Recruitment Drive Agent

Resume / profile screening · take-home evaluation · pipeline visibility. Replaces senior-engineer-as-screener model.

IN-HOUSE INTELLIGENCE LAYER

serves Product · Eng · QA · SRE · Support

RabbitView · Engineering Intelligence Platform

DORA metrics · individual/team velocity · product backlog · release monitoring · team analytics — for Product, Eng & QA

Report Portal

Centralized test reporting across all execution sources

TMS · AI-Enabled (Bedrock)

Duplicate / invalid-test detection coverage improvisations

Telemetry

Production signals routed into the quality feedback loop

Skill Matrix

Competency tracking across the 100+ engineer matrix

BUILT IN-HOUSE ON

TypeScript · Python · Docker · PostgreSQL · Redis

THE OPERATING MODEL

Buy commercial AI for breadth → Build custom agents for moats → Specs before agents
→ Visibility as a first-class deliverable → Customer impact is the only score that matters.

See full playbook: "Building AI Agents That Replaced ~95% of Manual QA Effort"

Madhu Sudhan Painoori

Senior Director, Quality Engineering · AutoRabit

p.mschary@gmail.com · +91 80086 90004

linkedin.com/in/madhu-sudhan-painoori · Hyderabad, India

© 2026 Madhu Sudhan Painoori

Read more in the chapters that follow — or use this page on its own for the 30-second walkthrough.

01. THE PROBLEM WE WERE SOLVING

Context

AutoRabit is the leading DevSecOps platform for Salesforce, with a multi-product suite — Release Management, CodeScan, Vault, DataOps — used by enterprise customers to govern, secure, and ship Salesforce changes at scale. During the period covered by this playbook, the platform was undergoing a monolith-to-microservices transformation with auto-scaling, meaning every QA assumption about environments, dependencies, and test isolation was simultaneously being rewritten.

The Manual QA Bottleneck

Like most growth-stage SaaS companies, our QA function had been built for an earlier era: human authors writing test cases from requirements, human engineers running regression suites, human triage on every flaky failure, and human review of every release-readiness signal. The math no longer worked:

- **Story-to-test latency was too long.** By the time a story was ready for QA, manually authored test cases lagged the developer's intent by days or weeks.
- **Regression coverage scaled linearly with effort.** Every new feature meant proportionally more manual maintenance — an unbounded liability.
- **Impact analysis was tribal knowledge.** Senior engineers "knew" what a code change affected. Newer engineers did not. Coverage gaps became escapes.
- **Performance issues surfaced late.** Profiling and scalability work happened reactively, often after a customer complaint.
- **Operational visibility was patchwork.** Quality KPIs lived in spreadsheets and tribal memory; leadership saw quality only after release-day surprises.
- **Recruiting drained senior engineers.** Screening rounds for QA hires consumed hours of senior bandwidth per hire, slowing org growth.

Why Traditional Automation Wasn't Enough

We had Selenium, TestNG, JMeter, Neoload, and an established automation discipline. None of it solved the upstream problem: authoring, maintaining, and reasoning about tests at the speed product development now demanded. A faster horse was not the answer. The answer was a different kind of vehicle.

02. THE STRATEGY — FROM TEST AUTOMATION TO AI-NATIVE QE

The shift we made — and that I would recommend to any QE leader being asked to do the same — was not "add AI to QA." It was redefining what QA is for in an AI-native engineering organization.

Five Principles That Guided Every Decision

- **Agents do the work. Humans set direction and judge outputs.** Wherever a workflow could be expressed as a specification, an agent owned execution. Engineers reviewed and steered.
- **Buy commercial AI for breadth, build custom agents for moats.** Copilot, ChatGPT, Claude, Bedrock, Cursor, KaneAI, Mabl, TestAI — these accelerate every engineer. But the platforms unique to our context — Jira-to-validated-release, performance optimization, recruitment screening — those we built ourselves.
- **Specs are the new code.** Using AWS Kiro, we drove agents from specs (requirements, design, tasks) rather than ad-hoc prompts. This gave us repeatability, reviewability, and the ability to evolve agent behavior the same way we evolve code.
- **Visibility is non-negotiable.** If leadership cannot see release readiness, test coverage, defect trends, velocity, and DORA metrics on demand, the AI investment becomes a black box. We built the visibility layer as a first-class deliverable.
- **Customer impact is the only score that matters.** Every initiative was tied to either escalation reduction, churn signal improvement, velocity gain, or engineering ROI. Vanity automation metrics — "% test cases automated" — were not goals.

The Operating Model

With a 10-person QA organization and a 100+ engineer matrix scope, we could not afford a centralized QE bottleneck. The operating model became: a small core team owned the agentic platform, the in-house tooling, and the testing standards. Embedded QE specialists worked alongside product engineering teams, with the agent fleet doing the heavy lifting. The Skill Matrix tool we built tracked competency across the entire matrix so we always knew where capability gaps were forming.

03. THE AI TOOL PORTFOLIO

Nine tools, each chosen for a specific job. The single biggest mistake organizations make is treating AI tooling as a uniform category. The tools below succeed in very different parts of the engineering loop, and the operating model for each is different.

TOOL	WHAT WE USE IT FOR
GitHub Copilot	In-IDE code generation and test scaffolding for QE engineers. Most consistent productivity lift for code authoring; team default.
ChatGPT	Quick reasoning, brainstorming, regex/SQL crafting, test data shaping. The universal scratchpad.
Claude	Long-context reasoning over specs, design docs, and code diffs. Strongest for impact analysis and structured outputs. Backbone of several agent prompts.
Amazon Bedrock	Production hosting for model access — enterprise-grade, region-aware, with the security guarantees our customers require.
AWS Kiro	Spec-driven agent development environment. Where our agentic platform was built — requirements, design, tasks all as Kiro specs, with trained skills, steering documents, and enabled MCPs.
Cursor	AI-first code editor used by engineers building the in-house platforms (TypeScript / Python). Step-change over Copilot-in-VSCode for refactoring and multi-file changes.
KaneAI	AI-native test authoring and management. Used to accelerate test design from natural language.
Mabl	AI-powered low-code automation for UI flows where Playwright was overkill and we wanted business-user contribution.
TestAI	AI-driven test automation — early adoption for self-healing scenarios before we standardized on Playwright + MCP + AI.

Selection Heuristic

For any team considering a similar portfolio: do not adopt every tool simultaneously. Start with the IDE assistant your engineers already want (usually Copilot or Cursor). Add a long-context model (Claude) for reasoning work. Get enterprise model hosting (Bedrock) in place before you touch production data. Layer specialized QA-focused tools (Mabl, KaneAI, TestAI) only after the foundation is real. The agentic platform sits on top of all of this — not in place of it.

04. THE FLAGSHIP — OUR AGENTIC AI PLATFORM

The platform consumes a Jira story and returns a validated release. Everything in between — story development, code analysis, test generation, execution — is owned by agents.

What It Does

The Agentic AI Platform is the single most important system we built. It collapses the traditional handoff chain — Product → Dev → QA → Release — into a continuous agent-mediated workflow. A story enters; a validated, tested change exits.

The Workflow

01 Jira Story Intake Agent pulls Jira story details — description, acceptance criteria, linked tickets, attached specs. Builds a structured understanding of intent.	02 Story Development Agent drives development of the story, producing or assisting code changes aligned to the acceptance criteria and existing system constraints.	03 Code-Change Detection Once code changes are committed, the platform captures the diff and triggers downstream analysis.
04 Impact Analysis Agent analyzes the diff against the broader codebase — which modules, flows, APIs, and customer scenarios are touched — to determine the test surface.	05 Test Case Generation Agent generates the specific test suite for the impacted surface area, including edge cases derived from the design and acceptance criteria.	06 Playwright + MCP + Bedrock Generated tests are automated through Playwright with MCP integration; execution results stream to Amazon Bedrock for AI analysis and self-healing on UI drift.
07 Result Validation Agent validates results against expected behavior, classifies failures (real defect vs. flaky vs. environment), and routes for human review where confidence is below threshold.	08 Release Signal Outcome flows into the Executive Monitoring Dashboard and Report Portal as a release-readiness signal — coverage delta, risk areas, blockers.	09 Feedback Loop Each cycle's outcomes feed back into the agent's steering documents, improving impact analysis accuracy and test selection over time.

How It Was Built

The platform was built on AWS Kiro with a spec-driven approach. Three artifact types live alongside the code:

- **Requirements specs** encode what the agent should accomplish, expressed at a level abstract enough to remain stable across model upgrades.
- **Design specs** describe how the agent reasons — the steps it takes, the inputs it consumes, the boundaries between agent decision and human review.
- **Task specs** are the executable unit. Each task is a discrete agent action; tasks compose into the larger workflow.

In addition to the specs, the platform required three less-visible investments that make the difference between an agent that demos well and an agent that runs in production:

- **Trained skills.** Domain-specific competencies — Salesforce metadata reasoning, AutoRabbit product semantics, Playwright DOM patterns — were trained into the agent so it does not have to discover them from prompts every run.
- **Steering documents.** The agent's behavioral guardrails, escalation rules, and quality thresholds live as steering documents alongside the specs. These are the things that change weekly as we learn what "good" looks like.
- **Enabled MCPs.** Model Context Protocol servers expose the tools the agent needs to act — Jira, the source repository, the test execution environment, the dashboards. The agent does not roam free; it acts only through curated MCP surfaces.

Why Spec-Driven Agents Won

The early temptation in any AI agent project is to write clever prompts. Clever prompts do not survive contact with reality: a model upgrade breaks them, a new edge case reveals an unstated assumption, the engineer who wrote them leaves. Specs survive. They are reviewable, versionable, testable, and inheritable. AWS Kiro made the spec-first discipline tractable; the discipline itself was the unlock.

KEY TAKEAWAY FOR QE LEADERS

If you are about to start an agentic platform initiative: invest 30% of the project budget in the spec, skill, and steering infrastructure before you build the first agent. Teams that skip this step ship a demo and then spend two years trying to operationalize it. Teams that invest here ship slower at first and pull ahead permanently around month four.

05. THE SPECIALIZED AGENT FLEET

Beyond the flagship platform, we built three purpose-built agents that solved expensive, recurring problems for the broader engineering organization. Two were aimed at QE; one extended the AI investment into the engineering team itself.

Performance Optimizing Agent

Performance work used to be reactive: a customer complained, a senior engineer dropped what they were doing, profiled, and patched. The Performance Optimizing Agent inverts that model.

- **Profiling on demand.** The agent runs profiling against a target service or flow, identifies the dominant cost centers — JVM, query, infrastructure, hot paths — and produces a structured report.
- **Optimization proposals.** Where the agent has high confidence (query-level tuning, index suggestions, configuration drift), it proposes specific changes. Where confidence is lower, it surfaces the hypothesis for engineer judgment.
- **Scalability assessment.** Before a release that touches load-bearing paths, the agent estimates scalability impact and recommends benchmark scenarios for JMeter / Neoload runs.

Net effect: performance work shifted left, profiling effort dropped, and the senior engineering bench reclaimed time that was previously consumed in production firefighting.

PR Quality & Coverage Agent

This is the agent that extended the AI investment from QE into the broader engineering organization. It activates on every pull request and acts as a quality gate before human review begins.

- **Per-PR coverage analysis.** The agent inspects each PR against the existing unit-test surface, identifies coverage gaps introduced by the change, and recommends specific test additions — not generic "add more tests" advice.
- **Code quality assessment.** Pattern analysis against the team's established standards — readability, complexity, defensive coding, error handling — with structured suggestions an engineer can accept, reject, or adapt.

This is the agent that explains why our matrix scope is 100+ engineers: the AI investment paid off across the entire engineering organization, not just QA. Coverage trends up over time as every merged PR carries the agent's improvements forward.

AutoScreen — Recruitment Drive Agent

Hiring a QE engineer used to consume four to six hours of senior engineer time per candidate. AutoScreen was built to minimize human effort for recruitment drives without compromising the signal we needed to make hiring decisions.

- **Resume and profile screening** with criteria-driven evaluation tied to the role's actual skill matrix.
- **Take-home and assessment evaluation** with rubric-aligned scoring and structured feedback engineers could review in minutes rather than hours.
- **Pipeline visibility** for hiring managers to see candidate progression without manual status-tracking.

THE PATTERN THAT WORKED

All three agents succeeded for the same reason: we picked workflows where the cost of the manual version was visible to leadership, the rubric for "good" was already articulated by a senior engineer, and the agent could be wrong without irreversible consequence. When choosing your next agent project, look for that combination first.

06. THE IN-HOUSE ENGINEERING INTELLIGENCE LAYER

Agents that work invisibly are agents that get shut down the first time something goes wrong. The platforms below are what made the AI investment durable — leadership could see the system, engineers could trust it, and customers benefited from the resulting predictability. Critically, this layer serves not just QE but the broader engineering organization: Product, Engineering, QA, SRE, and Support.

Everything in this section was built in-house on TypeScript, Python, Docker, PostgreSQL, and Redis.

RabbitView — Engineering Intelligence Platform

The cross-functional platform that explains why our matrix scope reaches 100+ engineers. RabbitView is the single pane of glass for Product, Engineering, and QA leadership — replacing the spreadsheet-and-Slack-status model that every growth-stage SaaS company eventually outgrows.

- **DORA metrics.** Deployment frequency, lead time for changes, change-failure rate, and mean time to restore — the four metrics that DevOps leaders are measured against — visible to every engineer and to leadership in real time.
- **Velocity tracking.** Individual and team velocity, with the discipline to use it as a leading indicator rather than a punitive measure. Teams see their own trajectory; leadership sees the aggregate.
- **Product backlog visibility.** What's coming, what's in flight, what's blocked — surfaced in the form that Product, Engineering, and QA can all act on without exporting to a deck.
- **Release monitoring and team analytics.** Release cadence, in-flight risks, post-release health, and team-level analytics that inform staffing, scope, and capability investments.

RabbitView is the asset that shifted my role from "QA leader who built some tools" to "engineering leader whose platforms power the organization." If you build only one in-house system, build this one.

Report Portal

The centralized test reporting and analytics surface. Every test run — Playwright, Mabl, KaneAI, agent-generated — flows through Report Portal. Engineers see failure history, flake patterns, and coverage trends in one place.

Test Management System (AI-Enabled)

A purpose-built TMS, AI-enabled through Amazon Bedrock, that does substantially more than commercial alternatives. Bedrock-powered semantic analysis surfaces duplicate and near-duplicate test cases across the corpus, flags invalid or stale tests whose acceptance criteria have drifted from product behavior, and recommends specific improvisations where coverage can be strengthened. The net effect is a meaningful reduction in test maintenance burden — fewer brittle tests, less duplicate effort, and higher signal-to-noise from every regression run.

Telemetry

Production signals routed back into the quality system. Datadog and Dynatrace are the source; Telemetry is the QE- and SRE-facing interpretation — anomalies, regressions, performance baselines, and the feedback loop into the agent's steering documents.

Skill Matrix

Competency tracking across the engineering matrix. Without it, you cannot operate an embedded QE model at 100+ engineers spanning Product, Engineering, QA, SRE, and Support. With it, we know where each capability lives, where it is thin, and how to staff every release confidently.

07. RESULTS, LESSONS, AND WHAT'S NEXT

Quantified Results

- **~95% of manual tedious QA effort automated.** Across test design, regression authoring, impact analysis, execution, and result triage, AI agents now do work that used to consume engineer hours.
- **~70% reduction in customer escalations.** The compounding effect of faster story-to-test latency, tighter impact analysis, and earlier performance work.
- **Churn signals tied to quality and deliverables improved.** Customer feedback shifted from quality-related complaints to feature-velocity requests — the right kind of pressure.
- **Recruiting throughput up; senior engineer hours down.** AutoScreen replaced the senior-engineer-as-screener model.

What Worked

- **Specs before agents.** Treating requirements, design, and tasks as first-class spec artifacts — not afterthought documentation — was the single highest-leverage decision.
- **Buy commercial AI, build the moat.** Resisting the urge to build from scratch what Copilot, Claude, and Bedrock do well, while investing heavily in the agentic platform that no vendor could provide for us.
- **Visibility as a first-class deliverable.** The Executive Monitoring Dashboard built leadership trust faster than any deck or update meeting could.
- **MCPs over free-roaming agents.** Constraining agents to act through curated MCP surfaces made debugging tractable and security defensible.

What Did Not Work — At First

- **Naive prompts.** Our first agents were prompt-engineered rather than spec-driven. They demoed beautifully and broke unpredictably. Moving to AWS Kiro specs took weeks; the productivity gain repaid the cost within a quarter.
- **Underestimating steering.** Behavior boundaries that we initially encoded in prompts belonged in steering documents. Once we separated them, agents became reviewable and improvable as a team.
- **Tool sprawl temptation.** We evaluated more AI tools than we adopted. The discipline to say "this one is not better than what we have" was harder than the discipline to adopt new tools.

What's Next

The current frontier is multi-agent coordination — multiple specialized agents collaborating on a single release, with the platform orchestrating handoffs and resolving conflicting signals. Adjacent work includes deeper integration of customer telemetry into the agent's prioritization logic, and extending the agentic pattern from QE into adjacent functions where the same conditions apply.

ABOUT THE AUTHOR

Madhu Sudhan Painoori

Senior Director, Quality Engineering — AutoRabit Pvt Ltd

Madhu is a Senior Director of Quality Engineering with 20+ years architecting and scaling QA functions across SaaS, product, and enterprise platforms. He currently leads Quality Engineering at AutoRabit, the leading DevSecOps platform for Salesforce, with a 10-person QA organization and a 100+ engineer matrix scope across cross-functional teams.

Prior to AutoRabit, Madhu led QA at CoMakeIT for Iddink (Netherlands), spent five and a half years at Oracle as a QA Lead across the WebCenter Sites and Cloud Services product lines, and led performance engineering, DBA, and security work at NetEnrich and Zeta Interactive. He is a Certified Professional Scrum Master and holds a B.Tech in Electronics and Communications Engineering from Vignan Institute of Technology and Science.

Madhu writes and speaks about AI-native quality engineering, agentic platforms, and the operating models that make them work.

OPEN TO CONVERSATIONS

Senior leadership opportunities in Quality Engineering, AI-driven engineering excellence, and developer productivity — particularly at SaaS, product, and AI-native companies. Advisory and fractional engagements considered.

Contact p.mschary@gmail.com · +91 80086 90004 · Hyderabad, India · linkedin.com/in/madhu-sudhan-painoori

© 2026 Madhu Sudhan Painoori. Personal playbook prepared for confidential reader use.